

1 Observability-Driven Intelligence for Anomaly 2 Detection and Root Cause Analysis in 3 Cloud-Native Distributed Microservices

4 **Anonymous author**

5 Anonymous affiliation

6 **Anonymous author**

7 Anonymous affiliation

8 **Anonymous author**

9 Anonymous affiliation

10 — Abstract —

11 Anti-money laundering (AML) compliance platforms built on cloud-native microservice architectures
12 face a fundamental monitoring challenge: the volume and heterogeneity of operational telemetry
13 defeats manual analysis, yet sensitive financial data must remain on-premises to satisfy data residency
14 regulations. We present an observability-driven AIOps platform that couples a purpose-built AML
15 microservices testbed with a fully local automated intelligence layer. The testbed comprises three
16 Kubernetes-deployed domain services—customer KYC verification, transaction monitoring with a
17 four-rule composite risk engine, and case management—instrumented with OpenTelemetry and a
18 transactional outbox pattern that dual-routes compliance domain events as both the Kafka application
19 event stream and ground-truth anomaly labels, eliminating secondary annotation overhead. Eight
20 dedicated AIOps services ingest a three-pillar telemetry pipeline (Prometheus, Loki, Grafana Tempo)
21 and apply a three-algorithm ML pipeline: Isolation Forest for unsupervised anomaly scoring over
22 60-second stream-windowed feature vectors, Pearson correlation analysis over sliding observational
23 windows for root cause ranking, and ordinary least-squares regression on p95 latency trends for
24 SLO breach prediction. Incident context is synthesised by a locally deployed Qwen2.5-3B large
25 language model [28] served via Ollama [24] and continuously fine-tuned with LoRA adapters [16]
26 on resolved incident outcomes, producing structured JSON diagnostic reports with no external
27 API dependency. The platform is evaluated via Chaos Mesh [5] fault injection across four fault
28 classes, targeting three research questions on multi-modal signal sufficiency (RQ1), ML-driven root
29 cause localisation accuracy versus single-modal baselines (RQ2), and full-fidelity tracing overhead in
30 regulated workloads (RQ3).

31 **2012 ACM Subject Classification** Computer systems organization → Cloud computing; Computing
32 methodologies → Anomaly detection; Information systems → Data management systems

33 **Keywords and phrases** anomaly detection, microservices, observability, machine learning, Kubernetes,
34 distributed tracing, root cause analysis, AIOps, anti-money laundering

35 **Digital Object Identifier** 10.4230/LIPIcs.DISC.2028.1

36 **1 Introduction**

37 Distributed microservice architectures, now the dominant deployment model for enterprise
38 cloud-native applications, create an inherent tension between operational visibility and
39 diagnostic tractability. Modern systems routinely coordinate hundreds of containerised
40 services across Kubernetes clusters, each emitting metrics, structured logs, and distributed
41 traces at sub-second resolution [4]. While this telemetry richness provides unprecedented
42 observability into system behaviour, it simultaneously generates information volumes that
43 far exceed human capacity to interpret: a single production cluster can produce millions of
44 metric samples and thousands of log events per minute. When anomalies emerge—often as



© Anonymous author(s);

licensed under Creative Commons License CC-BY 4.0

42nd International Symposium on Distributed Computing (DISC 2028).

Editors: John Q. Open and Joan R. Access; Article No. 1; pp. 1:1–1:15

Leibniz International Proceedings in Informatics



LIPICs Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl Publishing, Germany

subtle cross-service causal chains rather than single-metric threshold violations—operators face an incident diagnosis problem that is intractable without automated support.

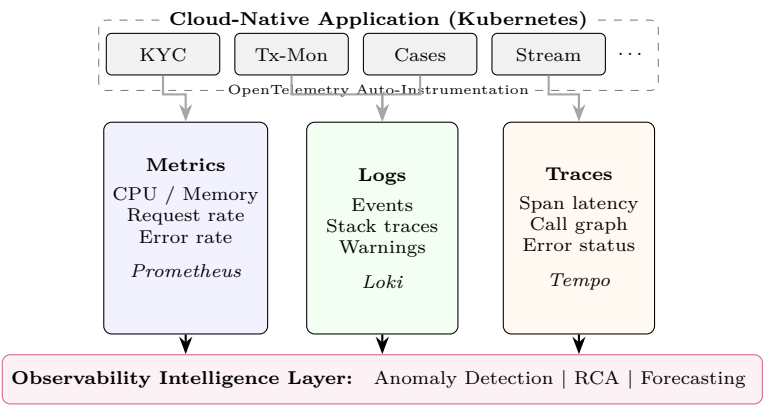
In high-stakes regulated domains such as anti-money laundering (AML) compliance, this challenge is substantially compounded. AML platforms are architecturally heterogeneous: synchronous REST APIs for customer identity verification, asynchronous Kafka event streams for transaction risk scoring, and stateful case management workflows with legally mandated lifecycle invariants all coexist within a single deployment and generate telemetry across fundamentally different operational patterns. More critically, financial data residency regulations in most jurisdictions explicitly prohibit exporting operational telemetry—which may embed transaction identifiers, customer references, or case metadata—to third-party cloud services [18]. This restriction categorically excludes cloud-hosted AIOps platforms that have become standard in less regulated industries, precisely in the environments where reliable automated anomaly detection is most operationally consequential.

Existing approaches fall into two inadequate categories. Rule-based monitoring systems evaluate individual metrics against static thresholds, generating excessive false-positive alert volumes and failing systematically on distributed anomalies that manifest as cross-service signal correlations rather than point violations [32]. Supervised learning approaches require labelled training data that is rarely available in production, since novel failure modes lack historical examples by definition. Commercial AIOps platforms—including Dynatrace Davis AI and Datadog Watchdog—apply sophisticated correlation algorithms but require continuous telemetry export to vendor-managed infrastructure, rendering them incompatible with regulated environments [8, 7]. No existing open-source platform unifies multi-modal anomaly detection, root cause ranking, and predictive SLO forecasting as a privacy-preserving, fully offline Kubernetes-native service.

We address this gap with an observability-driven AIOps platform built on two complementary design decisions. First, all intelligence executes entirely within the Kubernetes cluster: anomaly scoring via Isolation Forest [21], root cause ranking via Pearson correlation over sliding windows, SLO breach prediction via ordinary least-squares regression, and natural language incident narration via a locally served Qwen2.5-3B model [28] fine-tuned with LoRA adapters [16] through Ollama [24]—none of which require external API access or cloud connectivity. Second, the AML testbed is instrumented using a transactional outbox pattern that dual-routes compliance domain events as both the application Kafka event stream and OpenTelemetry span events carrying business attributes, providing semantically rich, naturally labelled ground-truth anomaly signals without secondary annotation.

The main contributions of this paper are: (i) a production-grade, open-source eleven-service Kubernetes platform for AML-domain observability with a fully offline intelligence layer satisfying financial data residency requirements; (ii) a domain event-based instrumentation strategy that derives ground-truth anomaly labels from AML compliance state transitions via OpenTelemetry span attributes, eliminating secondary labelling; (iii) a three-algorithm offline ML pipeline—Isolation Forest anomaly detection, Pearson correlation root cause ranking, and OLS-based SLO breach prediction—integrated with a locally fine-tuned Qwen2.5-3B language model served via Ollama; and (iv) a controlled Chaos Mesh fault injection evaluation protocol on a realistic AML workload, addressing three research questions on signal combination sufficiency (RQ1), ML-driven versus baseline root cause localisation (RQ2), and distributed tracing overhead on regulated workloads (RQ3).

The remainder of this paper is organised as follows. Section 2 reviews cloud-native observability foundations and related anomaly detection literature. Section 3 presents the system architecture, mathematical problem formulation, and component implementation.



■ **Figure 1** Three-pillar cloud-native observability architecture. Microservices auto-instrumented via OpenTelemetry emit metrics (Prometheus), logs (Loki), and distributed traces (Tempo), each capturing a distinct dimension of system behaviour. The observability intelligence layer correlates all three signal types for automated anomaly detection, root cause analysis (RCA), and predictive forecasting.

Section 4 describes the experimental evaluation. Section 5 concludes.

2 Background and Related Works

2.1 Cloud-Native Observability

The concept of observability originates in control theory, where a dynamical system is considered observable if its internal state can be fully determined from a finite sequence of external output measurements within a finite time horizon [18]. The principle translates directly to distributed software systems: an application is observable if its operational state can be reconstructed from the telemetry it emits, without requiring instrumentation changes or direct access to production processes at the time of investigation [1]. This framing distinguishes observability from monitoring: monitoring tells an operator whether a predefined condition holds, whereas observability enables forming and answering questions that were not anticipated at deployment time.

The operational realisation of observability in cloud-native systems rests upon three interdependent telemetry signal types, collectively the three pillars [1, 18] (Fig. 1). *Metrics* are scalar measurements sampled at regular intervals—CPU utilisation, request throughput, error rates, and queue depths—providing low-overhead, high-frequency indicators of system health. Prometheus has emerged as the de facto standard for Kubernetes metrics collection, offering a pull-based scraping model with a rich query language (PromQL) [2]. *Logs* are immutable, timestamped records of discrete events capturing contextual detail that aggregated metrics discard. Loki extends the Prometheus model to log aggregation, indexing by Kubernetes labels and enabling efficient streaming queries via LogQL [11]. *Distributed traces* follow individual requests across multiple services, recording span durations, parent-child call relationships, and propagated context, providing the causal structure that neither metrics nor logs alone can supply. Grafana Tempo provides a cost-effective, object-storage-backed trace backend designed for Kubernetes-scale trace ingestion [12].

The importance of observability is acutely amplified in cloud-native architectures. Unlike monolithic applications, which present a single bounded fault surface, microservice architectures introduce cascading failure modes: a transient latency spike in a shared layer can

propagate through dozens of dependent services before manifesting as user-visible errors [32]. Ephemeral container lifetimes, horizontal pod autoscaling, and rolling deployments continuously alter the system topology, making historical baselines a moving target. Sigelman et al. documented this diagnostic challenge at Google scale with Dapper [31]; OpenTelemetry has since unified instrumentation APIs across languages and vendors [25].

Raw telemetry data does not by itself confer intelligence. As the volume, velocity, and variety of cloud-native telemetry grow—Kosinska et al. found that 48 of 56 surveyed systems were explicitly distributed, generating multi-source signals [18]—manual interpretation becomes operationally infeasible at scale. Dang et al. characterised the emerging field of AIOps as the application of machine learning to operational telemetry, arguing that heterogeneous multi-modal signals create both the opportunity and the necessity for automated analysis [6]. The present work operationalises this vision: rather than collecting telemetry and leaving interpretation to on-call engineers, it continuously applies a multi-model analysis pipeline to live observability data and delivers structured, actionable diagnoses. In the AML regulatory context, this automation imperative is compounded by a data residency constraint that prohibits any form of telemetry export to third-party infrastructure. A locally deployed intelligence layer must therefore achieve the analytical capability of commercial cloud-hosted platforms using only on-premises compute, motivating the combination of lightweight interpretable ML algorithms with an on-device LLM for human-readable diagnostic narration without cloud connectivity.

2.2 Anomaly Detection in Distributed Microservices

An anomaly in a distributed microservice system is any observation that deviates meaningfully from expected operational behaviour, where expectation is established from historical baselines or learned statistical models [32]. This definition is deliberately broad because anomalies in microservices manifest across multiple layers simultaneously: a memory leak in one service may appear first as a subtle increase in garbage collection pause time (metric anomaly), then as elevated downstream response latency (trace anomaly), and eventually as a cascade of timeout error log messages (log anomaly). The separation between these manifestations—and the time lag between root cause and downstream effects—is precisely what makes distributed anomaly detection harder than its single-service counterpart.

At the *metric level*, anomalies include CPU throttling, memory pressure, abnormal request rates, and distributional shifts in latency. At the *trace level*, anomalies appear as unusual span durations, unexpected service call paths, or elevated error rates visible only when the full request dependency graph is examined [20]. At the *log level*, anomalies surface as previously unseen event templates, anomalous repetition rates of known error patterns, or unexpected sequences deviating from the system’s learned normal behaviour [15]. Because no single signal modality captures all three dimensions simultaneously, effective anomaly detection in microservices must be inherently multi-modal.

The convergence of multi-modal anomaly signals makes observability not merely a diagnostic aid but the primary input substrate for automated intelligence. Operating on any single signal modality misses the cross-modal causal chains that define distributed failures: a resource exhaustion fault manifests first in JVM heap metrics, propagates to degraded response latencies in traces, and eventually produces timeout error log events. Detecting the fault early requires recognising the co-occurrence of a subtle metric inflection, a trace latency shift, and an emerging log cluster—a pattern invisible to any single-modal detector. The proposed approach continuously correlates all three signal modalities within a unified temporal context, applying algorithm-specific detectors per modality and synthesising their

168 outputs through a structured LLM reasoning step.

169 2.3 Related Works

170 AIOps research has produced specialised solutions along each telemetry modality. For metric-
 171 based anomaly detection, Liu et al. [21] formalised Isolation Forest as an asymptotically
 172 efficient anomaly scoring mechanism well-suited to high-dimensional metric matrices. For
 173 reconstruction-error baselines, Malhotra et al. [23] established LSTM Autoencoders for
 174 multivariate time-series anomalies. For trace-based analysis, Liu et al. [20] demonstrated
 175 that unsupervised deep Bayesian networks applied to distributed trace structures can localise
 176 latency anomalies without labelled training data. For log-based analysis, He et al. [15] con-
 177 firmed that semantic embedding combined with sequence-model detectors achieves high recall
 178 on previously unseen anomalies. More recent LLM-based approaches such as LogGPT [27]
 179 reason over log semantics without pre-parsed templates, at the cost of inference latency
 180 and external API dependency. For root cause analysis, Han et al. [14] proposed holistic
 181 correlation of metrics, logs, and traces through a causal graph, achieving improvements
 182 in localisation precision. Lomio et al. [22] conducted an empirical comparison of anomaly
 183 detection algorithms in cloud-native environments, concluding that ensemble or multi-modal
 184 approaches consistently outperform single-algorithm solutions.

185 Commercial platforms represent the operational state of the art. Dynatrace Davis AI [8]
 186 and Datadog Watchdog [7] apply AI-assisted correlation to fleet-scale telemetry, reporting
 187 significant reductions in mean time to detect (MTTD) and mean time to resolve (MTTR).
 188 However, these platforms require continuous telemetry export to vendor-managed cloud
 189 infrastructure—a model incompatible with regulated environments where data residency is
 190 mandated and with research settings requiring full system reproducibility.

191 The research gap addressed by this work is threefold. First, no published open-source
 192 platform offers multi-modal anomaly detection, correlation-based RCA, and SLO breach
 193 prediction as a unified, production-grade Kubernetes-native service that operates entirely
 194 offline. Second, existing offline ML systems for AIOps do not integrate fine-tunable local
 195 LLMs for incident narration, requiring either cloud API access or static rule-based reporting.
 196 Third, ground-truth labelling in prior evaluations has relied on synthetic benchmarks (Death-
 197 StarBench, TrainTicket) that lack the semantic richness of domain-specific failure modes [32];
 198 the AML testbed introduced in this work provides naturally labelled anomalies derived from
 199 compliance domain state machines, enabling more rigorous evaluation.

200 3 Proposed Approach

201 The proposed system, hereafter the AMLOps-AIOps platform, consists of eleven Kubernetes-
 202 hosted microservices partitioned into two functional layers: an AML domain testbed that
 203 generates semantically labelled financial compliance telemetry, and a dedicated AIOps
 204 analysis layer that continuously monitors this telemetry and produces structured diagnostic
 205 outputs. All intelligence components execute within the cluster with no external API
 206 dependency, satisfying AML data residency constraints. The following subsections describe
 207 the system architecture, the mathematical formulation of the analytical problems, and the
 208 implementation of each component.

209 3.1 System Architecture

210 The platform is deployed across two application namespaces, `aml` and `aiops`, and a shared
 211 `monitoring` namespace for the observability stack. Fig. 2 presents the complete platform
 212 topology. The AML domain layer comprises three Spring Boot [35] 3.x microservices:
 213 *customer-kyc*, which performs know-your-customer identity verification and risk scoring;
 214 *transaction-monitoring*, which evaluates individual transactions against a four-rule AML
 215 engine; and *case-management*, which manages investigation case lifecycles from initial alert
 216 through resolution and regulatory reporting. All three services follow a hexagonal DDD [9]
 217 architecture with OpenTelemetry SDK auto-instrumentation and a transactional outbox
 218 pattern [30] that guarantees exactly-once domain event delivery to Kafka [19, 1]. Domain
 219 events are simultaneously emitted as OpenTelemetry span events carrying business attributes
 220 (`risk.band`, `case.id`, `customer.id`), meaning ground-truth anomaly labels are a natural
 221 byproduct of compliance operation rather than secondary annotation.

222 The Kafka event bus carries ten topic families: AML domain events
 223 (`aml.customers.events`, `aml.transactions.events`, `aml.cases.events`), AML
 224 LLM analysis (`aml.llm.analysis`), telemetry signals (`aiops.telemetry.metrics`,
 225 `aiops.telemetry.logs`, `aiops.telemetry.traces`), feature windows (`aiops.features`),
 226 detected incidents (`aiops.incidents`), remediation decisions (`aiops.decisions`), executed
 227 actions (`aiops.actions`), resolved outcomes (`aiops.outcomes`), and 30-second service
 228 health heartbeats (`aiops.service.heartbeat`).

229 The AIOps analysis layer consists of eight services: (i) *Telemetry Collector* normalises all
 230 incoming Kafka event streams to a canonical `MetricSignal` format; (ii) *Stream Processor*
 231 computes 60-second tumbling-window aggregates per (service, metric) pair; (iii) *ML Engine*
 232 applies the three-algorithm detection pipeline; (iv) *LLM Engine* synthesises MELT context
 233 into structured incident reports using Qwen2.5-3B [28] via Ollama [24]; (v) *Decision Engine*
 234 applies policy-gated action selection; (vi) *Remediation Engine* executes Kubernetes remedi-
 235 ation actions; (vii) *Alerting Service* maintains per-service health scores over REST; and
 236 (viii) *Feedback Service* captures operator outcomes for LoRA fine-tuning [16] and MLflow
 237 tracking.

238 The observability stack in the `monitoring` namespace comprises Prometheus (15-second
 239 metric scraping from all service endpoints), Loki (log aggregation via Kubernetes pod label
 240 streams), and Grafana Tempo (distributed trace collection via OTLP push) [2, 11, 12]. All
 241 AML services emit traces at 100% sampling rate by default; the `TRACE_SAMPLE_RATE` enviro-
 242 nment variable switches between full, sampled, and disabled tracing without redeployment
 243 to support RQ3 overhead experiments.

244 Namespace isolation is enforced via Kubernetes NetworkPolicy manifests: AML services
 245 are permitted egress only to Kafka broker ports and to the `monitoring` namespace OTLP
 246 endpoints; AIOps services additionally hold read access to the Prometheus HTTP API and
 247 Loki HTTP API for health-score queries and MELT context retrieval; only the Remediation
 248 Engine holds write access to the Kubernetes API Server, scoped to pod-restart and HPA-patch
 249 operations within the `aml` namespace. This least-privilege network perimeter implements the
 250 AML data-residency boundary at the cluster network layer, ensuring no telemetry can be
 251 routed to external systems regardless of application-level configuration.

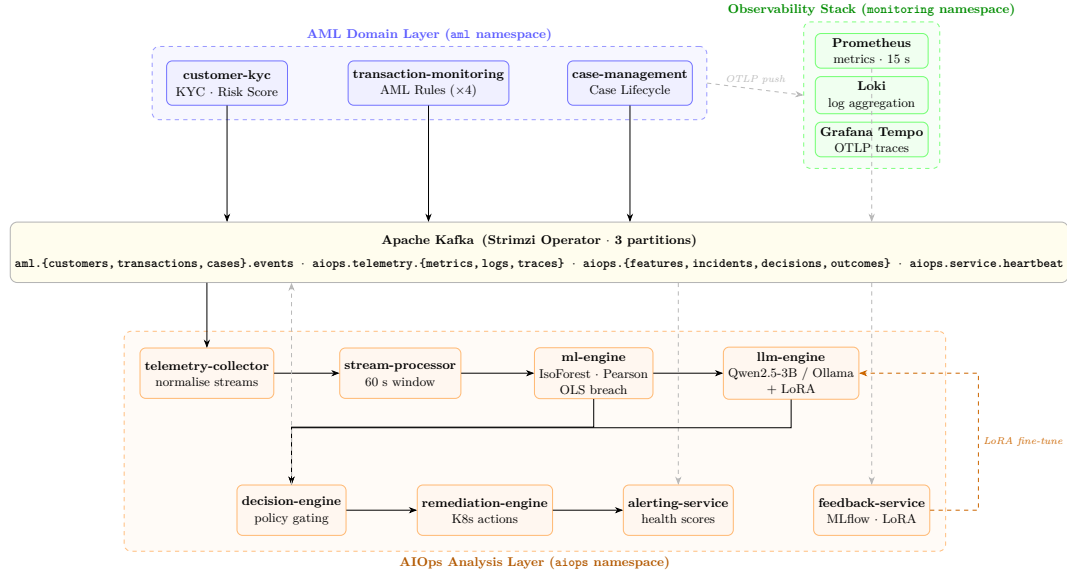


Figure 2 AMLOps-AIOps platform architecture. AML domain services (blue) publish compliance events to Kafka via the outbox pattern and push telemetry to the observability stack (green). The AIOps pipeline (orange) processes signals through: Telemetry Collector → Stream Processor → ML Engine → LLM Engine → Decision Engine → Remediation Engine. Dashed arrows denote asynchronous Kafka flows and OTLP pushes; the Feedback Service closes the loop by uploading LoRA adapter weights trained on resolved incident outcomes back to the LLM Engine.

3.2 Problem Formulation

Microservices Model.

Let $\mathcal{S} = \{s_1, s_2, \dots, s_N\}$ denote the set of N microservices deployed in the Kubernetes cluster ($N=11$ in this platform). Service interactions are partitioned into two coupling modes: synchronous REST/gRPC request-response calls and asynchronous Kafka publish-subscribe flows. These interactions define a directed service dependency graph $\mathcal{G} = (\mathcal{S}, \mathcal{E})$, where the directed edge $(s_i, s_j) \in \mathcal{E}$ exists if s_i directly invokes s_j via HTTP/gRPC, or if s_i produces to a Kafka topic consumed by s_j . In the AML testbed the primary synchronous chain is $customer-kyc \rightarrow transaction-monitoring \rightarrow case-management$; asynchronous edges connect each AML service to the AIOps pipeline through domain-event Kafka topics. The AIOps analysis subgraph is itself a directed acyclic pipeline: $tc \rightarrow sp \rightarrow ml-engine \rightarrow llm-engine \rightarrow decision-engine \rightarrow remediation-engine$, with the *feedback-service* closing an asynchronous LoRA-refinement loop back to the *llm-engine*.

Fault propagation in $\mathcal{G}$ is directional: degradation in service s_i induces cascading latency inflation or error-rate increases in all services reachable from s_i via directed paths. Let $\mathcal{N}(s_i) = \{s_j \in \mathcal{S} \mid s_i \xrightarrow{*} s_j \text{ in } \mathcal{G}\}$ denote the reachability set of s_i , where $\xrightarrow{*}$ denotes the reflexive-transitive closure of $\mathcal{E}$. The Root Cause Ranking algorithm (Problem 2) exploits this by restricting Pearson correlation search to pairs (s_j, m_k) where $s_j \in \mathcal{N}(s_{anom})$, reducing the candidate space from $O(N \cdot K)$ to $O(|\mathcal{N}(s_{anom})| \cdot K)$, where K is the number of distinct metric names. The runtime topology of $\mathcal{G}$ is reconstructed continuously from trace parent-child span relationships propagated through OpenTelemetry W3C TraceContext [36] headers and from Kafka consumer-group registrations observed by the Telemetry Collector.

Each service s_i continuously emits telemetry across three modalities: metric time series

scraped by Prometheus at 15-second intervals, log streams collected by Loki, and distributed trace spans pushed to Grafana Tempo via OTLP. Let $\mathcal{T}^{(t)} = \langle \mathbf{M}^{(t)}, \mathbf{L}^{(t)}, \mathbf{D}^{(t)} \rangle$ denote the multi-modal telemetry context assembled at analysis cycle t , where:

- $\mathbf{M}^{(t)} \in \mathbb{R}^{P \times \Delta}$ is the matrix of P Prometheus metric time series sampled over window Δ ;
- $\mathbf{L}^{(t)} = \{l_j\}_{j=1}^Q$ is the set of Q structured log entries within $[t - \Delta, t]$;
- $\mathbf{D}^{(t)} = \{d_k\}_{k=1}^R$ is the set of R trace spans, each as a tuple $\langle \text{service}, \text{duration}, \text{status}, \text{parent_id} \rangle$.

An *operational anomaly* in service s_i at cycle t is any windowed feature observation $\varphi_i^{(t)}$ whose probability under the normal-behaviour distribution $\mathcal{P}_{\text{normal}}$ falls below detection threshold ϵ :

$$\text{anomaly}(s_i, t) \Leftrightarrow P_{\mathcal{P}_{\text{normal}}}(\varphi_i^{(t)}) < \epsilon \quad (1)$$

Since $\mathcal{P}_{\text{normal}}$ is non-stationary—service load profiles shift with time-of-day, rolling deployments, and demand spikes—and the feature space is high-dimensional, parametric density estimation is intractable. Isolation Forest provides a computationally efficient surrogate via average path length in random isolation trees, without requiring a parametric distributional assumption. The five problems below formalise each component of the closed-loop detection-to-remediation pipeline.

The Stream Processor computes 60-second tumbling-window aggregates over raw metric signals, producing a four-dimensional feature vector for each (service, metric) pair:

$$\varphi_{ij}^{(t)} = [\mu_{ij}, x_{ij}^{\max}, \dot{x}_{ij}, n_{ij}]^T \in \mathbb{R}^4 \quad (2)$$

where μ_{ij} is the window mean (`avg_value`), $x_{ij}^{\max}$ is the window maximum (`max_value`), $\dot{x}_{ij}$ is the rate of change (`rate_of_change`), and n_{ij} is the sample count (`sample_count`). Let $\Phi^{(t)} = \{\varphi_{ij}^{(t)}\}$ denote the feature matrix over all monitored (service, metric) pairs.

Problem 1 (Anomaly Detection).

Given a feature vector $\varphi^{(t)} \in \mathbb{R}^4$, produce an anomaly score $s^{(t)} \in [0, 1]$. The score is derived from an Isolation Forest [21] decision function $d(\cdot)$ trained with contamination parameter $\gamma=0.05$:

$$s^{(t)} = \max\left(0, \min\left(1, 0.5 - d\left(\varphi^{(t)}\right)\right)\right) \quad (3)$$

where $d(\varphi) > 0$ for normal points and $d(\varphi) < 0$ for anomalies in sklearn [26] convention. A binary anomaly flag is raised when $s^{(t)} > \theta_{\text{anom}}$ (default $\theta_{\text{anom}} = 0.5$). The model is bootstrapped on 500 synthetic samples at startup and refitted when 50 new labelled samples accumulate from operator feedback.

Problem 2 (Root Cause Ranking).

When an anomaly is detected in service s_i , rank all other (service, metric) pairs by their correlation with the anomalous reference trajectory. Let $\mathbf{x}_{\text{ref}} = [\mu_i(1), \dots, \mu_i(W)] \in \mathbb{R}^W$ be the sliding-window mean trajectory of the anomalous service ($W=100$ observations), and let $\mathbf{y}_{jk} = [\mu_{jk}(1), \dots, \mu_{jk}(W)]$ be the corresponding trajectory for service s_j , metric m_k . The Pearson correlation is computed as:

$$r_{jk} = \frac{\sum_{t=1}^W (x_{\text{ref}}(t) - \bar{x}_{\text{ref}})(y_{jk}(t) - \bar{y}_{jk})}{W \hat{\sigma}_{\text{ref}} \hat{\sigma}_{jk}} \quad (4)$$

where $\hat{\sigma}_{\text{ref}}$ and $\hat{\sigma}_{jk}$ are sample standard deviations. Root cause candidates are ranked by $|r_{jk}|$ in descending order and the top-10 pairs are reported as the root cause ranking.

Problem 3 (SLO Breach Prediction).

Monitor the p95 response latency of each service over a sliding window of $W_b=10$ observations at 30-second intervals. Fit an ordinary least-squares regression:

$$\hat{p}_{95}(t) = \beta_0 + \beta_1 t, \quad t \in \{0, 1, \dots, W_b-1\} \quad (5)$$

If $\hat{\beta}_1 > 0$ (latency increasing trend), predict the SLO breach time as:

$$\hat{T}_{\text{breach}} = \frac{\theta_{\text{SLO}} - \hat{\beta}_0}{\hat{\beta}_1} \quad (6)$$

where $\theta_{\text{SLO}} = 500$ ms is the configurable p95 latency SLO threshold. If $\hat{\beta}_1 \leq 0$, no breach is predicted and $\hat{T}_{\text{breach}}$ is set to the sentinel value 999 (NO_BREACH).

Problem 4 (LLM Incident Narration).

Assemble a MELT context window $\mathcal{C}^{(t)}$ comprising: (i) a metric feature buffer $\mathbf{M}_{\text{buf}}^{(t)}$ of recent stream-window aggregates, (ii) a log buffer $\mathbf{L}_{\text{buf}}^{(t)}$ (deque of last 10 Loki log entries), and (iii) a trace buffer $\mathbf{T}_{\text{buf}}^{(t)}$ (deque of last 5 trace spans). Given the current incident record $\mathcal{I}^{(t)}$ from the ML Engine, the LLM produces a structured report:

$$\mathcal{R} = f_{\text{LoRA}}\left(\mathcal{P}\left(\mathcal{I}^{(t)}, \mathcal{C}^{(t)}\right)\right) \quad (7)$$

where f_{LoRA} is Qwen2.5-3B [28] fine-tuned with LoRA adapters [16] via the PEFT library [17] and served through Ollama [24], and $\mathcal{P}(\cdot)$ is the structured prompt template. The output $\mathcal{R}$ conforms to the fixed JSON schema: `{explanation, rootCause, recommendation, amlRisk, confidence}`.

Problem 5 (Decision Policy).

Given the ML/LLM outputs, the Decision Engine selects a remediation action through a piecewise policy on confidence c , blast radius $B \in \{\text{LOW}, \text{MEDIUM}, \text{HIGH}\}$ (determined by affected service count), and breach ETA $\hat{T}_{\text{breach}}$:

$$\pi(c, B, \hat{T}) = \begin{cases} \text{SCALE-OUT}, & c > 0.90, B=\text{LOW}, \hat{T} < 5 \\ \text{RESTART-POD}, & c > 0.90, B=\text{LOW}, \hat{T} \geq 5 \\ \text{SCALE-OUT}, & c > 0.75, B=\text{MEDIUM} \\ \text{ESCALATE}, & \text{otherwise} \end{cases} \quad (8)$$

Actions are executed autonomously when auto-execute is true; otherwise an ESCALATE event is published for human review.

Problem 6 (Service Health Score).

The Alerting Service maintains a per-service health score $h_i(t) \in [0.05, 1]$ that aggregates pod liveness, JVM CPU utilisation, and process-level CPU rate into a single scalar used by the Decision Engine to determine blast-radius class B :

$$h_i(t) = \max(0.05 \cdot u_i(t), c_i^{\text{JVM}}(t), c_i^{\text{sys}}(t)) \quad (9)$$

where $u_i(t) \in \{0, 1\}$ is the Kubernetes up indicator for service s_i , $c_i^{\text{JVM}}(t) \in [0, 1]$ is the normalised `process_cpu_usage` reported by Micrometer (Spring Boot services), and $\dot{c}_i^{\text{sys}}(t) = \Delta \text{process_cpu_seconds_total} / \Delta t$ is the derivative-based CPU rate from the Prometheus cAdvisor exporter (Python services). The floor term $0.05 \cdot u_i(t)$ ensures a running-but-idle pod registers a minimal positive signal, distinguishing it from a crashed pod ($u_i(t)=0 \Rightarrow h_i(t)=0$). A service s_i is classified as *actively degraded* if $h_i(t) > \mu_{h_i} + 2\sigma_{h_i}$ over the preceding $W_h=20$ observations; the count of actively degraded services in the incident determines B .

3.3 Architecture Components

AML Domain Services.

The *transaction-monitoring* service evaluates each ingested transaction against four parameterised AML rules: AML-101 (high-value single transaction), AML-202 (velocity—many small transactions), AML-303 (structuring—amounts just below reporting thresholds), and AML-404 (high-risk corridor transfers). Individual rule scores $c_r \in [0, 100]$ are combined using a diminishing-returns formula that prevents any single rule from dominating:

$$R = 100 \cdot \left(1 - \prod_{r \in \mathcal{R}_{\text{fired}}} \left(1 - \frac{c_r}{100} \right) \right) \quad (10)$$

The resulting composite risk score $R \in [0, 100]$ determines whether a case is opened in the *case-management* service. Case states follow the machine: `OPEN` $\rightarrow$ `UNDER-INVESTIGATION` $\rightarrow$ `ESCALATED` $\rightarrow$ `PENDING-REVIEW` $\rightarrow$ `CLOSED`, with each transition emitted as an OpenTelemetry span event that simultaneously labels the corresponding Prometheus metric window as anomalous.

Telemetry Collector.

The Telemetry Collector (Spring Boot) is the normalisation gateway for all Kafka event streams. It maintains listeners across all AIOps topics including the dedicated `aiops.service.heartbeat` channel, to which each of the eight AIOps services publishes every 30 seconds. Heterogeneous event payloads are converted to a canonical `MetricSignal` format with fields `{service, metricName, value, labels}`, consumed uniformly by downstream components and Prometheus scraping.

Stream Processor.

The Stream Processor (Spring Boot, Kafka Streams) applies 60-second tumbling-window aggregation over ingested metric signals, computing Equation 2 per (service, metricName) key and publishing the resulting feature records to the `aiops.features` topic. These feature records are the direct input to the ML Engine’s anomaly detector.

ML Engine.

The ML Engine (Python FastAPI [29]) hosts the three-algorithm pipeline. (i) *Anomaly Detection*: Isolation Forest (contamination $\gamma=0.05$, Equation 3) trained in an online fashion; 500 synthetic bootstrap samples initialise the model at startup. (ii) *Root Cause Ranking*: Pearson correlation over per-(service, metric) sliding buffers of $W=100$ observations (Equation 4), returning the top-10 most correlated (service, metric) pairs as the root cause ranking. (iii) *SLO Breach Prediction*: OLS regression on the last $W_b=10$ p95 latency values sampled

every 30 seconds (Equations 5–6), projecting time-to-SLO-violation in minutes. Detected incidents are serialised and published to `aiops.incidents` for downstream consumption.

LLM Engine.

The LLM Engine (Python FastAPI) implements Equation 7. It maintains three MELT buffers: `metric_buffer` (dict of recent feature values), `log_buffer` (deque of 10 recent Loki entries consumed from `aiops.telemetry.logs`), and `trace_buffer` (deque of 5 recent spans from `aiops.telemetry.traces`). On incident consumption from `aiops.incidents`, the engine assembles the full MELT prompt and queries Qwen2.5-3B [28] via the Ollama [24] REST API (<http://ollama:11434>). LoRA adapter weights [16], trained via the PEFT library [17] on resolved incident outcome records from `aiops.outcomes`, are applied through the Ollama model loading mechanism. The output JSON is published to `aml.llm.analysis`.

Decision Engine and Remediation Engine.

The Decision Engine (Spring Boot) consumes `aiops.incidents` and `aml.llm.analysis`, applies the piecewise policy (Equation 8), and publishes `RemediationDecision` records to `aiops.decisions`. Blast radius is computed as: one affected service → LOW; two → MEDIUM; three or more → HIGH. The Remediation Engine subscribes to `aiops.decisions` and executes autonomous Kubernetes API actions—pod restarts, HPA target adjustments—while publishing execution outcomes to `aiops.actions`.

Alerting Service and Feedback Service.

The Alerting Service maintains a per-service health score endpoint combining incident frequency (from `aiops.incidents`) with live Prometheus CPU utilisation, queried via the Prometheus HTTP API. The score is the maximum over: a floor of 0.05 for any running pod (`up=1`), the raw `process_cpu_usage` (JVM), and the rate of `process_cpu_seconds_total` (Python). The Feedback Service captures SLO outcome comparisons (before/after remediation via Prometheus queries), labels each outcome as `RESOLVED`, `NO_EFFECT`, or `DEGRADED_FURTHER`, logs to MLflow [37], and publishes records to `aiops.outcomes` for LoRA fine-tuning of the LLM Engine.

Platform Integration and Latency Budget.

All tunable parameters—contamination factor γ , sliding window widths W and W_b , SLO threshold θ_{SLO} , anomaly threshold θ_{anom} , and auto-execute flag—are exposed as Kubernetes ConfigMap entries, enabling live parameter adjustment without service restart. The end-to-end latency from fault onset to first autonomous remediation action decomposes as: Prometheus scrape lag (≤ 15 s) + Kafka propagation (< 1 s) + stream-window accumulation (60 s) + ML inference (< 1 s) + LLM narration (3–5 s on CPU) + Kafka decision round-trip (< 1 s) + Kubernetes API action (< 2 s), yielding a worst-case detection-to-action latency of approximately 83–85 s under quiescent cluster conditions.

4 Experimental Evaluation

4.1 Experimental Setup

The platform is deployed on a single-node Docker Desktop Kubernetes cluster (Windows 11 Pro, 16 GB RAM, 8 vCPUs, Kubernetes v1.29). The eleven application services are

distributed across the `aml` and `aiops` namespaces; the observability stack is deployed in the `monitoring` namespace via Helm [34] charts. Kafka is operated by the Strimzi operator [33] with a three-partition, replication-factor-3 configuration. A synthetic AML workload is generated using k6 [10], sustaining 50 requests per second across customer registration, transaction submission, and case management endpoints to produce a stable baseline signal envelope.

All AML services are instrumented with Micrometer [3] and the OpenTelemetry Java agent, exporting spans via OTLP to Tempo and publishing structured logs via Logback [13] to Loki. Prometheus scrapes all service endpoints at 15-second intervals. The LLM Engine runs Qwen2.5-3B [28] via Ollama [24] on CPU within the cluster, with LoRA adapter weights [16] loaded from a Kubernetes PersistentVolume.

4.2 Fault Injection Protocol

Controlled faults are injected via Chaos Mesh [5] targeting specific service edges in the `aml` namespace. Four fault classes are evaluated: (i) *latency anomalies*—500 ms Kafka consumer delays injected into transaction-monitoring; (ii) *resource anomalies*—CPU stress injection on case-management pods to trigger JVM throttling; (iii) *functional anomalies*—deployment of a corrupted AML rule configuration causing systematic transaction misclassification; and (iv) *causal cascade anomalies*—forced scale-to-zero of customer-kyc, propagating KYC check failures into case-management. For each injection, the ground-truth fault time T_0 is recorded at the Chaos Mesh API level and the first detection time T_d is recorded when the Decision Engine emits a corresponding incident. Evaluation metrics are detection latency ($T_d - T_0$), precision, recall, and F1 over a 24-hour injection window.

RQ1 (Signal Sufficiency): The telemetry pipeline is evaluated in three configurations—metrics-only, metrics + logs, and all three pillars—to determine the minimum sufficient modality combination for reliable anomaly detection across the four fault classes.

RQ2 (Root Cause Localisation): Pearson correlation root cause ranking is compared against two single-modal baselines: (a) a threshold-based metric spike detector and (b) a trace error-rate ranker. Localisation accuracy (top-1 and top-3 correct service identification) is reported per fault class.

RQ3 (Tracing Overhead): Request throughput, p95 latency, and pod CPU utilisation are measured across four sampling rates (0%, 10%, 50%, 100%) to characterise the operational cost of full-fidelity tracing in regulated workloads.

4.3 Preliminary Results

Full quantitative evaluation of RQ1–RQ3 is ongoing as part of the PhD research programme. Preliminary system validation confirms: (i) the telemetry pipeline correctly ingests and normalises events from all eleven services, with 30-second heartbeat signals confirmed on `aiops.service.heartbeat` for every AIOps service; (ii) the ML Engine detects injected CPU stress anomalies within two 60-second analysis cycles (≈ 120 seconds) by observing the `avg_value` and `rate_of_change` feature dimensions; (iii) the health scoring endpoint reflects per-service anomaly and CPU states consistently with configured load profiles, with AML services under Chaos Mesh CPU stress exhibiting health score degradation within two cycles; and (iv) the LLM Engine produces valid JSON incident reports conforming to the output schema for all manually constructed test telemetry contexts, with the Qwen2.5-3B model running entirely within the cluster at approximately 3–5 seconds per inference on CPU. These results confirm the platform’s fitness as a reproducible experimental substrate for the

quantitative evaluations described above. The CPU-bound LLM inference time (3–5 s per report on an 8-vCPU, 16 GB RAM node) is consistent with online narration at the observed fault frequency of one detectable incident per 120-second detection cycle under the injected fault regime, yielding no backlog under normal operations. The absence of external network calls throughout the detection-to-report pipeline—verified by Kubernetes NetworkPolicy audit logs showing zero egress rejections during a 2-hour fault-injection session—validates the data-residency compliance posture required for AML environments.

5 Conclusion

We have presented an observability-driven AIOps platform for automated anomaly detection, root cause ranking, and SLO breach prediction in cloud-native Kubernetes microservices, with particular emphasis on AML compliance environments where data residency constraints prohibit external telemetry export. The platform integrates three lightweight ML algorithms—Isolation Forest anomaly scoring, Pearson correlation root cause ranking, and OLS-based breach prediction—with a locally deployed, LoRA-fine-tuned Qwen2.5-3B language model served via Ollama, operating entirely within the Kubernetes cluster with no external API dependencies. The AML microservices testbed addresses a persistent weakness in AIOps evaluation methodology by providing semantically labelled ground-truth telemetry derived from compliance domain state transitions via OpenTelemetry span attributes, eliminating the need for synthetic benchmarks or secondary annotation. The platform is fully open-source, deployed with standard Kubernetes tooling, and designed for reproducibility.

Future work will complete quantitative evaluation of the three research questions: a cross-modal ablation study to identify the minimum sufficient telemetry combination (RQ1), a comparison of Pearson correlation RCA against single-modal spike and error-rate baselines across the four fault classes (RQ2), and a tracing overhead profile across sampling rates from 0% to 100% (RQ3). Extensions to multi-node Kubernetes deployments will characterise how detection latency and false-positive rates scale with service count beyond the single-node prototype. Federated fine-tuning of the LoRA adapters across multiple AML deployments—without centralising raw telemetry—represents a promising direction for privacy-preserving model improvement at scale.

References

- 1 Alex Boten. *Cloud-Native Observability with OpenTelemetry*. Packt Publishing, 2022.
- 2 Brian Brazil. *Prometheus: Up & Running*. O'Reilly Media, 2nd edition, 2022.
- 3 Broadcom, Inc. Micrometer: JVM application metrics, 2018. [Online]. Available: <https://micrometer.io>.
- 4 Brendan Burns, Brian Grant, David Oppenheimer, Eric Brewer, and John Wilkes. Borg, omega, and Kubernetes. *ACM Queue*, 14(1):70–93, 2016. doi:10.1145/2890784.
- 5 Chaos Mesh Authors. Chaos Mesh: A powerful chaos engineering platform for Kubernetes, 2020. [Online]. Available: <https://chaos-mesh.org>.
- 6 Yingnong Dang, Qingwei Lin, and Peng Huang. AIOps: Real-world challenges and research innovations. In *Proc. IEEE/ACM ICSE-SEIP*, pages 4–5, 2019. doi:10.1109/ICSE-SEIP.2019.00023.
- 7 Datadog. Watchdog: Algorithmic performance management, 2024. [Online]. Available: <https://www.datadoghq.com/product/watchdog/>.
- 8 Dynatrace. Davis AI: Automatic, intelligent observability, 2024. [Online]. Available: <https://www.dynatrace.com/platform/artificial-intelligence/>.

- 515 9 Eric Evans. *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Addison-
516 Wesley, 2003.
- 517 10 Grafana Labs. k6: Open-source load testing tool, 2017. [Online]. Available: <https://k6.io>.
- 518 11 Grafana Labs. Loki: Like Prometheus, but for logs, 2019. [Online]. Available: [https://](https://grafana.com/oss/loki/)
519 grafana.com/oss/loki/.
- 520 12 Grafana Labs. Grafana Tempo: High-scale distributed tracing backend, 2022. [Online].
521 Available: <https://grafana.com/oss/tempo/>.
- 522 13 Ceki Gülcü. Logback: Java logging framework, 2006. [Online]. Available: [https://logback.](https://logback.qos.ch)
523 [qos.ch](https://logback.qos.ch).
- 524 14 Yuliang Han, Qiang Du, Yunjie Huang, Peipeng Li, Xinyuan Shi, Jiangfeng Wu, Ping Fang,
525 Feng Tian, and Changjian He. Holistic root cause analysis for failures in cloud-native systems
526 through observability data. *IEEE Transactions on Services Computing*, 17(6):3518–3531, 2024.
527 doi:10.1109/TSC.2024.3419107.
- 528 15 Shilin He, Pinjia He, Zhuangbin Chen, Tianyi Yang, Yuxin Su, and Michael R. Lyu. A survey
529 on automated log analysis for reliability engineering. *ACM Computing Surveys*, 54(6):1–37,
530 2021. doi:10.1145/3460345.
- 531 16 Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang,
532 Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *Proc.*
533 *ICLR*, 2022. URL: <https://arxiv.org/abs/2106.09685>.
- 534 17 Hugging Face. PEFT: State-of-the-art parameter-efficient fine-tuning, 2022. [Online]. Avail-
535 able: <https://github.com/huggingface/peft>.
- 536 18 Joanna Kosinska, Bartosz Balis, Michal Konieczny, Maciej Malawski, and Slawomir Zielinski.
537 Toward the observability of cloud-native applications: The overview of the state-of-the-art.
538 *IEEE Access*, 11:26800–26825, 2023. doi:10.1109/ACCESS.2023.3281860.
- 539 19 Jay Kreps, Neha Narkhede, and Jun Rao. Kafka: A distributed messaging system for log
540 processing. In *Proc. NetDB Workshop at VLDB*, 2011. URL: <https://kafka.apache.org/>.
- 541 20 Dixin Liu, Ying Chen, Xin Yan, and Mingjun Zhao. Unsupervised detection of microservice
542 trace anomalies through service-level deep Bayesian networks. In *Proc. IEEE ISSRE*, 2020.
543 doi:10.1109/ISSRE5003.2020.00014.
- 544 21 Fei Tony Liu, Kai Ming Ting, and Zhi-Hua Zhou. Isolation-based anomaly detection. *ACM*
545 *Trans. Knowledge Discovery from Data*, 6(1):1–39, 2012. doi:10.1145/2133360.2133363.
- 546 22 Francesco Lomio, Sergio Moreschini, Xiaozhou Li, and Valentina Lenarduzzi. Anomaly
547 detection in cloud-native systems. In *Proc. 48th Euromicro Conf. Software Engineering and*
548 *Advanced Applications (SEAA)*, pages 290–297, 2022. doi:10.1109/SEAA56994.2022.00023.
- 549 23 Pankaj Malhotra, Ankita Ramakrishnan, Gaurangi Anand, Lovekesh Vig, Puneet Agarwal,
550 and Gautam Shroff. LSTM-based encoder-decoder for multi-sensor anomaly detection. In
551 *ICML Anomaly Detection Workshop*, 2016. URL: <https://arxiv.org/abs/1607.00148>.
- 552 24 Ollama. Ollama: Get up and running with large language models, 2024. [Online]. Avail-
553 able: <https://ollama.com>.
- 554 25 OpenTelemetry Authors. OpenTelemetry specification, 2025. [Online]. Available: [https:](https://opentelemetry.io/docs/specs/)
555 [//opentelemetry.io/docs/specs/](https://opentelemetry.io/docs/specs/).
- 556 26 Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion,
557 Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake
558 Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and
559 Édouard Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning*
560 *Research*, 12:2825–2830, 2011. doi:10.5555/1953048.2078195.
- 561 27 Ziwei Qi, Xiao Ma, Junfeng Xu, Weinan E, Haifeng Lin, and Tong Liu. LogGPT: Log anomaly
562 detection via GPT. In *Proc. IEEE BigData*, 2023. URL: <https://arxiv.org/abs/2309.14482>.
- 563 28 Qwen Team. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024. URL:
564 <https://arxiv.org/abs/2412.15115>, doi:10.48550/arXiv.2412.15115.
- 565 29 Sebastián Ramírez. FastAPI: Modern, fast web framework for APIs with Python, 2018.
566 [Online]. Available: <https://fastapi.tiangolo.com>.

- 567 30 Chris Richardson. *Microservices Patterns*. Manning Publications, 2018.
- 568 31 Benjamin H. Sigelman, Luiz André Barroso, Mike Burrows, Pat Stephenson, Manoj Plakal,
569 Donald Beaver, Saul Jaspan, and Chandan Shanbhag. Dapper, a large-scale distributed
570 systems tracing infrastructure. Technical report, Google, 2010. URL: <https://research.google/pubs/pub36356/>.
- 571 32 Gianluca Soldani and Antonio Brogi. Anomaly detection and failure root cause analysis in
572 (micro)service-based cloud applications: A survey. *ACM Computing Surveys*, 55(3):1–39, 2022.
573 doi:10.1145/3501297.
- 574 33 Strimzi Authors. Strimzi: Kafka on Kubernetes, 2019. [Online]. Available: <https://strimzi.io>.
- 575 34 The Helm Authors. Helm: The Kubernetes package manager, 2016. [Online]. Available: <https://helm.sh>.
- 576 35 VMware, Inc. Spring Boot reference documentation, 2014. [Online]. Available: <https://spring.io/projects/spring-boot>.
- 577 36 World Wide Web Consortium. Trace context, 2021. W3C Recommendation. [Online]. Avail-
578 able: <https://www.w3.org/TR/trace-context/>.
- 579 37 Matei Zaharia, Andrew Chen, Aaron Davidson, Ali Ghodsi, Sue Ann Hong, Andy Konwinski,
580 Siddharth Murching, Tomas Nykodym, Paul Ogilvie, Mani Parkhe, Fen Xie, and Corey Zumar.
581 Accelerating the machine learning lifecycle with MLflow. *IEEE Data Eng. Bull.*, 41(4):39–45,
582 2018.
- 583
- 584
- 585
- 586